

소프트웨어 설계서

대중교통 사용 내역 관리 앱 개발

문서 버전 : v1.2

작성일 : 2026-09-14

소속 : 컴퓨터공학부 컴퓨터공학전공 2학년

작성자 : 김영재

문서 개요

항목	내용
문서명	소프트웨어 설계서
시스템명	대중교통 사용 내역 관리 앱 개발
문서 버전	v1.2
상태	작성 중
작성일	2026-09-14

목차

1. 설계 개요	6
1.1. 목적	6
1.2. 범위	6
1.3. 용어 및 약어	6
1.4. 참고 문서	6
2. 요구사항 및 설계 원칙	7
2.1. 요구사항 추적	7
2.2. 설계 원칙	7
3. 시스템 아키텍처	8
3.1. 시스템 구성도	8
3.2. 구성 요소	8
3.3. 주요 처리 흐름	8
4. 상세 설계	10
4.1. 기능/모듈 목록	10
4.2. 모듈 상세 설계	10
4.3. 상태 및 시퀀스	12
5. 데이터 설계	13
5.1. 데이터 저장 구조 및 순서	13
5.2. 저장 데이터 정의	13
5.3. 데이터 정합성 및 오류 방지	14
6. 인터페이스 설계	15
6.1. 내부 모듈 연계 목록	15
6.2. 함수 정의	15
7. 화면 및 사용자 경험 설계	17
7.1. 화면 목록	17

7.2. 화면 설계	18
8. 비기능 요구사항 설계	22
9. 배포 및 운영 설계	23
9.1. 배포 구조 (실행 방법)	23
9.2. 장애 대응	23
10. 테스트 설계	24

1. 설계 개요

1.1. 목적

이 시스템은 대중교통(지하철·버스) 이용 시 후불 교통카드만으로는 파악하기 어려운 세부 지출 내역을 사용자가 직접 등록·관리하고, 기간별 지불 금액을 계산해 볼 수 있는 웹 애플리케이션이다. 이 설계서는 제안요청서 및 제안서(기획 및 요구사항 정의서)에서 확정한 기능 요구사항(FR-01~FR-06)과 예외 처리 요구사항(EH-01~EH-03)을 실제 구현 가능한 수준의 모듈 구조, 데이터 구조, 화면 흐름으로 구체화하는 것을 목적으로 한다.

1.2. 범위

구분	포함 범위	제외 범위	비고
기능	FR-01~FR-06 전체 (요금데이터/템플릿등록/내역등록/ 화면구현/지불금액계산/수정삭제)	회원가입·로그인, 다중 사용자 지원, 외부 결제 연동	
시스템	웹 브라우저 기반 단일 페이지 애플리케이션(HTML/CSS/JS)	서버, 데이터베이스, 외부 API 연동	외부 라이브러리 사용 금지(제약)
데이터	브라우저 LocalStorage에 저장되는 JSON 데이터 (요금/템플릿/사용내역)	서버 DB, 클라우드 동기화	

1.3. 용어 및 약어

용어/약어	정의	비고
FR	기능 요구사항 (Functional Requirement)	제안요청서 FR-01~06
EH	예외 처리 요구사항 (Exception Handling)	EH-01, 02는 발주자 정의 / EH-03은 개발팀 자체 제안
TransferDC	환승 할인율 (Transfer Discount). 기본요금에 곱해 환승 요금을 계산하는 값	0이면 무료 환승, 0.1이면 10%만 지불
LocalStorage	브라우저에 키-값 형태로 데이터를 영속 저장하는 Web Storage API	탭을 닫아도 데이터 유지
JSON	JavaScript Object Notation, 데이터 저장/교환 형식	

1.4. 참고 문서

문서명	버전/일자	출처 또는 링크	비고
제안요청서	2026-09-08	정유운 발주	FR-01~06, EH-01~02 최초 정의
기획 및 요구사항 정의서(제안서)	2026-09-08	김영재 작성	구현 방안, 데이터 구조, EH-03 추가 제안

2. 요구사항 및 설계 원칙

2.1. 요구사항 추적

요구사항 ID	요구사항 내용	설계 반영 위치	검증 방법	상태
FR-01	요금 데이터 지정	4.1 FareModule / 5장 데이터 설계	콘솔 로그로 요금 객체 확인	미착수
FR-02	노선 템플릿 등록	4.1 TemplateModule / 7장 화면 설계	등록 후 템플릿 목록 반영 확인	미착수
FR-03	대중교통 사용 내역 등록	4.1 LogModule	템플릿 구간 수 = 생성된 내역 수 확인	미착수
FR-04	화면 구현	4.1 UIModule / 7장 화면 설계	버튼 클릭 시 화면 갱신 확인	미착수
FR-05	지불 금액 계산	4.1 PaymentCalcModule / 6.2 함수 정의	수기 계산값과 비교	미착수
FR-06	템플릿·내역 수정 및 삭제	4.1 TemplateModule, LogModule	삭제/수정 후 재조회로 반영 확인	미착수
EH-01	잘못된 입력 방지	7.2 화면 설계(버튼/선택 기반 입력), trim()	임의 입력 테스트	미착수
EH-02	데이터 저장/로드 오류 방지	4.1 StorageModule(try-catch)	저장 중 강제 오류 주입 테스트	미착수
EH-03	내역 0건일 때 지불 금액 계산 예외 (자체 제안)	4.2 PaymentCalcModule 예외 처리	빈 내역 상태에서 계산 실행	미착수

2.2. 설계 원칙

- 모든 사용자 입력은 버튼·선택 목록(select) 기반으로 제공하여 잘못된 입력이 원천적으로 발생하지 않도록 한다 (EH-01).
- 저장·로드 로직은 모두 try-catch로 감싸 오류가 발생해도 이전 데이터를 그대로 보존한다 (EH-02).
- 요금 데이터는 지역이 늘어나도 코드 수정 없이 객체에 데이터만 추가하면 되는 구조로 설계한다 (확장성).
- 발주자가 정의하지 않은 예외 상황(EH-03 등)은 개발팀이 자체적으로 식별하고 문서화하여 설계에 반영한다.
- 모든 변수·함수에는 기능 설명 주석을 반드시 작성한다 (비기능 요구사항).

3. 시스템 아키텍처

3.1. 시스템 구성도

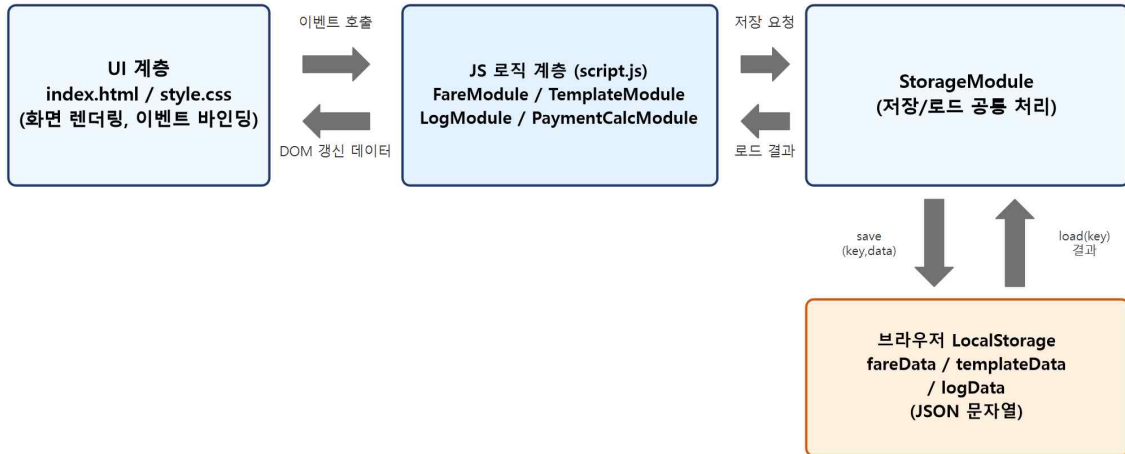


그림 1 시스템 구성도 (UI 계층 ↔ JS 로직 계층 ↔ LocalStorage)

3.2. 구성 요소

구성 요소	역할	기술/버전	배포 단위	연계 대상
UI 계층	화면 렌더링, 이벤트 바인딩	HTML5 / CSS3	정적 파일(index.html , style.css)	JS 로직 계층
JS 로직 계층	요금 계산, 데이터 CRUD, 이벤트 처리	순수 JavaScript (ES6, 외부 라이브러리 미사용)	단일 파일(script.js)	UI 계층, StorageModule
StorageModule	요금/템플릿/내 역 데이터 영속 저장·로드	Web Storage API (LocalStorage)	브라우저 내장 기능	JS 로직 계층

3.3. 주요 처리 흐름

순서	처리 단계	입력	출력	오류/예외
1	노선 템플릿 등록	노선명/교통수단/지 역/요금등급/환승여 부 선택	템플릿 목록에 즉시 반영	선택형 입력이라 오류 없음(EH-01)
2	사용 내역 등록	등록된 템플릿 선택	사용 내역 목록에 구간 수만큼 추가	템플릿 미선택 시 등록 버튼 비활성화
3	지불 금액 계산	조회 단위(월/년) 또는 개별 내역 체크	총 지불 금액 화면 표시	대상 0건이면 0원 반환(EH-03)

4	템플릿/내역 수정·삭제	수정/삭제 버튼 클릭	목록에서 즉시 반영/제외	저장 실패 시 이전 데이터 유지(EH-02)
---	-----------------	-------------	------------------	-----------------------------

4. 상세 설계

4.1. 기능/모듈 목록

모듈 ID	모듈명	주요 책임	입력/출력	의존 모듈
MOD-01	FareModule	지역·교통수단·요금등 급별 요금 및 TransferDC 관리 (FR-01)	입력: 없음 / 출력: 지역별 요금 객체	없음
MOD-02	TemplateModule	노선 템플릿 등록·수정·삭제 (FR-02, FR-06)	입력: 사용자 선택값 / 출력: 템플릿 배열	FareModule
MOD-03	LogModule	사용 내역 등록·수정·삭제 (FR-03, FR-06)	입력: 템플릿 또는 직접 입력값 / 출력: 사용 내역 배열	TemplateMod ule
MOD-04	UIModule	화면 렌더링, 이벤트 바인딩 (FR-04)	입력: 각 모듈의 데이터 / 출력: DOM 갱신	TemplateMod ule, LogModule, PaymentCalc Module
MOD-05	PaymentCalcModule	지불 금액 계산 (FR-05)	입력: 내역 배열, 요금 데이터, 조회 단위 / 출력: 총 지불 금액	FareModule, LogModule
MOD-06	StorageModule	LocalStorage 저장·로드 공통 처리 (EH-02)	입력: 저장 키·데이터 / 출력: 로드된 객체	없음 (모든 모듈이 호출)

4.2. 모듈 상세 설계

항목	내용
모듈명	FareModule
목적	지역·교통수단별 기본 요금과 환승 할인율을 관리한다.
호출 조건	앱 최초 실행 시 1회(초기 데이터 없을 때)
처리 절차	코드 내에 지역별 요금 객체를 정의 → 지역 추가 시 배열에 항목만 추가
예외 처리	해당 없음(내장 데이터)
성능 기준	-
로그/모니터링	-

항목	내용
모듈명	TemplateModule
목적	사용자가 자주 쓰는 노선을 템플릿으로 등록해 재사용한다.
호출 조건	템플릿 등록/수정/삭제 버튼 클릭 시
처리 절차	선택값을 객체로 구성 → 템플릿 배열에 추가 → StorageModule.save() 호출
예외 처리	필수 선택값 누락 시 등록 버튼 비활성화
성능 기준	-
로그/모니터링	-

항목	내용
모듈명	LogModule
목적	템플릿 또는 직접 입력을 사용 내역으로 저장한다.
호출 조건	내역 등록/수정/삭제 버튼 클릭 시
처리 절차	템플릿의 구간(TP List)을 순회하며 내역 객체 생성 → 배열에 추가 → 저장
예외 처리	이름 앞뒤 공백은 trim() 처리(EH-01)
성능 기준	-
로그/모니터링	-

항목	내용
모듈명	PaymentCalcModule
목적	선택된 기간·내역에 대한 총 지불 금액을 계산한다.
호출 조건	지불 금액 계산 버튼 클릭 시
처리 절차	대상 내역별로 FareModule에서 기본요금 조회 → IsTransfer=true면 요금×TransferDC 적용 → 합산
예외 처리	대상 내역이 0건이면 0원 반환(EH-03)
성능 기준	계산 결과가 화면에 표시되는 시점까지
로그/모니터링	해당 없음(개인 프로젝트 범위)

항목	내용
모듈명	StorageModule
목적	모든 모듈의 저장/로드를 공통으로 처리한다.
호출 조건	각 모듈에서 데이터 변경 시마다 호출
처리 절차	JSON.stringify 후 저장 / JSON.parse 후 반환, 전 구간 try-catch로 감쌘
예외 처리	오류 발생 시 콘솔 로그 출력 후 이전 데이터 유지(EH-02)
성능 기준	-
로그/모니터링	저장/로드 실패 시 콘솔 경고 로그

4.3. 상태 및 시퀀스



그림 2 노선 템플릿 기반 사용 내역 등록 플로우차트 (FR-02 → FR-03)

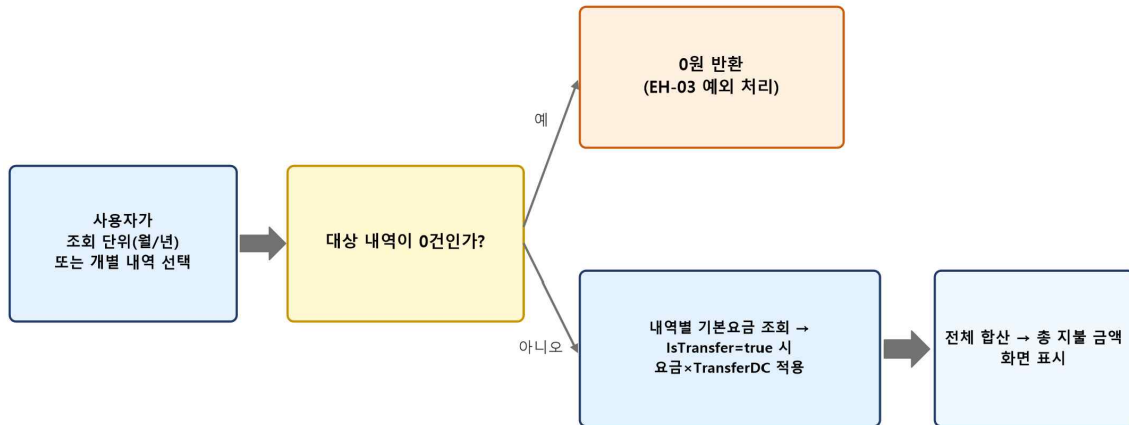


그림 3 지불 금액 계산 플로우차트 (FR-05, EH-03)

5. 데이터 설계

5.1. 데이터 저장 구조 및 순서

모든 데이터는 브라우저 LocalStorage에 3개의 키로 JSON 문자열 형태로 저장된다. 저장 및 로드 순서는 다음과 같다.

- ① 앱 최초 실행 시 - FareModule이 fareData 키의 존재 여부를 확인하고, 없으면 코드에 내장된 기본 요금 객체를 1회 저장한다.
- ② 앱 실행 시마다 - StorageModule이 fareData / templateData / logData 3개 키를 모두 로드하여 각각의 런타임 변수(in-memory)에 대입한다.
- ③ 템플릿 등록 시 - templateList 배열에 항목을 추가한 뒤 templateData 키로 다시 저장한다(write-through).
- ④ 사용 내역 등록 시 - logList 배열에 항목을 추가한 뒤 logData 키로 다시 저장한다.
- ⑤ 수정·삭제 시 - 해당 배열에서 항목을 변경/삭제한 뒤 같은 키로 다시 저장하여 항상 LocalStorage와 런타임 데이터를 일치시킨다.

저장 키 (LocalStorage)	파일 저장 형식	런타임 자료구조	설명
fareData	JSON Object (Region 배열)	let fareData = { Region: [...] }	지역별 교통수단·요금등급별 요금 및 TransferDC
templateData	JSON Object (Template 배열)	let templateList = [...]	사용자가 등록한 노선 템플릿 목록
logData	JSON Object (Log 배열)	let logList = [...]	실제 사용 내역 목록 (템플릿 기반 자동 생성 또는 직접 입력)

5.2. 저장 데이터 정의

구조	설명	식별 키	주요 필드	비고
fareData.Region[]	지역별 요금표	Name (지역명)	Train/Bus 각각 { Child, Youth, Adult, TransferDC }	코드 내장, 지역 추가 시 배열에 항목만 추가
templateData.Template[]	노선 템플릿	Name (템플릿명)	TP List: [{ Name, Type, Region, AgeType, IsTransfer }]	사용자가 화면에서 등록
logData.Log[]	사용 내역	Time (등록 시각)	Name, Type, Region, AgeType, IsTransfer	템플릿 등록 시 구간 수만큼 자동 생성, 직접 등록도 가능

5.3. 데이터 정합성 및 오류 방지

항목	정책/설계 내용	검증 방법
입력 정합성	노선명 등 문자열 입력 시 trim()으로 앞뒤 공백 제거, 나머지 항목은 버튼/선택 목록으로만 입력받아 오입력 자체를 차단 (EH-01)	임의 공백 포함 입력 테스트
저장/로드 오류 방지	JSON.stringify / JSON.parse를 try-catch로 감싸 오류 시 이전 데이터를 그대로 유지 (EH-02)	저장 도중 강제 오류 주입 테스트
백업/복구	별도 백업 기능 없음 - 브라우저 데이터(LocalStorage) 삭제 시 복구 불가 (개인 프로젝트 범위 밖, 향후 JSON 내보내기 기능으로 개선 가능)	해당 없음

6. 인터페이스 설계

6.1. 내부 모듈 연계 목록

연계 ID	호출 모듈	대상 모듈	호출 함수	설명
IF-01	UIModule	TemplateModule	addTemplate(templateObj)	노선 템플릿 등록
IF-02	UIModule	LogModule	addLogFromTemplate(templateName)	템플릿 기반 사용 내역 일괄 등록
IF-03	UIModule	PaymentCalcModule	calculatePayment(targetLogs, fareData)	지불 금액 계산
IF-04	TemplateModule / LogModule	StorageModule	save(key, data) / load(key)	LocalStorage 저장/로드

6.2. 함수 정의

항목	내용
함수명	calculatePayment(targetLogs, fareData)
호출 위치	UIModule (지불 금액 계산 버튼 클릭 시)
입력 파라미터	targetLogs: 계산 대상 사용 내역 배열 / fareData: 요금 데이터 객체
반환 값	숫자 (총 지불 금액, 원 단위)
처리 로직	① 각 내역의 지역·교통수단·요금등급으로 fareData에서 기본요금 조회 → ② IsTransfer가 true면 기본요금 × TransferDC 적용 → ③ 전체 합산
예외 처리	targetLogs가 빈 배열이면 0을 반환 (EH-03)

항목	내용
함수명	addLogFromTemplate(templateName)
호출 위치	UIModule (템플릿으로 내역 등록 버튼 클릭 시)
입력 파라미터	templateName: 선택한 템플릿 이름
반환 값	boolean (성공 여부)
처리 로직	① templateList에서 templateName으로 템플릿 조회 → ② TP List를 순회하며 구간마다 내역 객체 생성 → ③ logList에 추가 후 StorageModule.save() 호출
예외 처리	템플릿을 찾지 못하면 false 반환, 화면에 안내

항목	내용
함수명	save(key, data) / load(key)
호출 위치	TemplateModule, LogModule, FareModule 등 모든 데이터 변경 지점

입력 파라미터	key: 저장 키 문자열 / data: 저장할 객체
반환 값	save: boolean(성공 여부), load: 파싱된 객체 또는 null
처리 로직	JSON.stringify(data) → localStorage.setItem(key, ...) / localStorage.getItem(key) → JSON.parse(...)
예외 처리	try-catch로 감싸 오류 시 false 또는 null 반환, 콘솔에 오류 로그 출력 (EH-02)

7. 화면 및 사용자 경험 설계

7.1. 화면 목록

화면 ID	화면명	진입 방법	주요 기능	비고
UI-01	메인 화면 (대시보드)	앱 최초 실행 시 기본 진입	총 지불 금액 요약, 등록 템플릿 수·이번 달 내역 건수, 최근 내역 미리보기, 각 화면 바로가기	다른 화면으로 이동하는 시작점
UI-02	노선 템플릿 등록 화면	상단 탭 [템플릿 등록]	노선명 입력 + 교통수단/지역/요금등급/환 승여부 선택, 등록된 템플릿 목록 관리(구간 수 표시)	버튼/선택 기반 입력만 허용 (EH-01)
UI-03	사용 내역 목록 화면	상단 탭 [사용 내역]	내역 조회, 항목별 체크 선택, 개별 수정·삭제	템플릿으로 일괄 등록된 내역도 함께 조회
UI-04	지불 금액 계산 화면	상단 탭 [지불 계산]	조회 단위(월/년) 또는 선택 내역 기준 계산, 노선별 상세 내역(기본요금/환승적용/금액) 표시	대상 0건일 때 '0원' 표시 (EH-03)

7.2. 화면 설계



그림 4 UI-01 메인 화면(대시보드) 목업

대중교통 사용 내역 관리

대시보드

템플릿 등록

사용 내역

지불 계산

새 템플릿 등록

[노선명 입력]

[교통수단 ▼] [지역 ▼] [요금등급 ▼]

[환승여부 □] (템플릿에 구간 추가)

등록된 템플릿 목록

템플릿명	구간 수	관리
Test (대구 3호선 → 724번 버스)	2 구간	[내역등록][삭제]
출근길	1 구간	[내역등록][삭제]

그림 5 UI-02 노선 템플릿 등록 화면 목업



그림 6 UI-03 사용 내역 목록 화면 목업

대중교통 사용 내역 관리

대시보드
템플릿 등록
사용 내역
지불 계산

지불 금액 계산

[이번 달 ▼] 또는 [선택한 내역만] (계산하기)

총 지불 금액

42,300 원

상세 내역

노선명	기본요금	환승 적용	금액
대구 3호선	2,000원	-	2,000원
724번 버스	1,500원	× 10%	150원

※ 대상 내역이 0건이면 오류 대신 '0원'으로 표시 (EH-03)

그림 7 UI-04 지불 금액 계산 화면 목업

8. 비기능 요구사항 설계

영역	목표/기준	설계 및 구현 방안	측정 방법
유지보수성	코드 가독성 확보	모든 변수·함수에 기능 설명 주석 작성 (제안서 3.4)	코드 리뷰 체크리스트
확장성	지역·요금 추가가 쉬워야 함	요금 데이터를 배열 기반 객체로 설계하여 지역 추가 시 데이터만 삽입하면 되도록 구성	신규 지역 추가 테스트
안정성	예외 상황에도 프로그램이 멈추지 않아야 함	버튼/선택 기반 입력(EH-01), try-catch 기반 저장/로드(EH-02), 0건 예외 처리(EH-03)	강제 오류 주입 테스트
호환성	주요 브라우저에서 정상 동작	순수 JavaScript(외부 라이브러리 미사용), 표준 Web Storage API만 사용	Chrome/Edge 등에서 실행 확인

9. 배포 및 운영 설계

9.1. 배포 구조 (실행 방법)

- ① 최종 산출물 폴더(index.html, style.css, script.js)를 다운로드하고 압축을 해제한다.
- ② index.html 파일을 Chrome 등 웹 브라우저로 더블클릭하여 연다. (별도 설치, 웹서버 구동, 인터넷 연결 불필요)
- ③ 최초 실행 시 기본 요금 데이터가 자동으로 LocalStorage에 저장된다.
- ④ 이후 화면에서 노선 템플릿 등록 → 사용 내역 등록 → 지불 금액 계산 순서로 기능을 사용한다.
- ⑤ README.md에 실행 방법과 기능 설명을 함께 제공한다 (제안서 6장 최종 산출물 계획).

9.2. 장애 대응

항목	내용
데이터 저장 실패	저장/로드 중 오류 발생 시(EH-02) 콘솔에 오류 로그를 출력하고 이전 데이터를 그대로 유지한다. 개인 프로젝트 범위상 별도 사용자 알림 팝업은 구현하지 않는다.
브라우저 데이터 삭제	사용자가 브라우저 데이터를 직접 삭제하면 LocalStorage가 초기화되며 복구할 수 없다. 별도 백업 기능은 이번 범위에서 제외하며, 향후 개선 사항으로 기록한다.

10. 테스트 설계

테스트 유형	대상/범위	입·출력 / 검증 기준	완료 기준
기능 테스트	FR-01 ~ FR-06 전체	각 기능이 명세대로 정상 동작하는가	전체 항목 통과
데이터 영속성 테스트	LocalStorage 저장/로드	브라우저 탭을 닫았다 다시 열어도 데이터가 유지되는가	데이터 유지 확인
예외 처리 테스트	EH-01, EH-02, EH-03	잘못된 입력·저장 오류·내역 0건 상황에서 프로그램이 멈추지 않는가	3개 시나리오 모두 통과
시나리오 테스트 1	환승(IsTransfer=true) 포함 템플릿 등록 → 내역 등록	등록된 내역 수 = 템플릿 구간 수 일치 확인	수 일치
시나리오 테스트 2	TransferDC=0.1 환승 내역 포함 월 단위 지불 금액 계산	수기 계산값과 비교	오차 없음
시나리오 테스트 3	템플릿/사용 내역 삭제 후 재조회	목록에서 정상적으로 제외되는가	정상 제외 확인